I. Proposal

Tracking The Attendance of Students: The Class Monitor

by Ashton Zac G. de Veyra and Sean Lester C. Villarin

Manual attendance-taking can be exhausting and inefficient, especially when repeated by class monitors or teachers. It can lead to errors such as incorrect records, missed names, or duplicate entries, and organizing the data afterward can also be time-consuming. This process also reduces valuable class time that could be used for more productive activities. Furthermore, manual systems often lack proper organization, making it difficult to retrieve or review attendance records when needed.

To solve this problem, this project introduces a QR code-based attendance system that automates the process of recording attendance. The system aims to reduce effort, improve accuracy, and make attendance tracking faster and more convenient.

Project's Main Objectives:

- **Automate attendance recording using a QR code scanning system, eliminating manual tracking.**
- **Improve accuracy by preventing duplicate entries and minimizing human errors.**
- **Provide a fast, convenient, and user-friendly interface for recording attendance.**
- **Classify and manage attendance records based on time (early, on time, late).**
- **Enable easy searching, organization, and export of attendance data into usable formats.**

Program's Features:

1. QR Code Generation

- Students can enter their name to be used as data for QR generation via the *qrcode* function
- When generated, the data is encoded in *JSON* format and can be used for QR generation

2. QR Code Scanning

- Students can register attendance quickly by scanning their QR code via a webcam.
- Automatically records the student's name, time, and attendance status.
- The webcam used for scanning is opened via *OpenCV* and is decoded through *JSON*

3. Image Snapshot

- Captures a real-time image of the user at the exact moment of time-in, to prove the authenticity of the student scanning
- Triggered instantly upon successful QR decryption via *OpenCV*
- Saves the image in an organized folder

4. Attendance Status Classification

- Determines if a student is "Early," "On time," or "Late," and whether an admission slip is required.
- Applies color-coded status in the interface for easy visualization
- The *datetime* feature was used for time comparison

5. Searchable Attendance Table

- All registered students are displayed in a scrollable table
- A search bar is also available to find/search for a specific student
- These features are made possible via Python's *tkinter* library, mainly used for the GUI

6. Report Exporting

- Generates attendance reports in CSV (Excel) and txt formats via *csv*
- Reports are automatically saved with the date of registration included in the filename, using the *datetime* module for record-keeping

7. Admin Mode

- Ability to modify and delete specific records as the admin
- Requires a login via a password, which should only be known by the administrator
- Uses file system management to allow the administrator to also change the password and save it as a txt file in an organized folder

8. User-Friendly Interface

- Simple and intuitive GUI with buttons, legends, and prompts
- A color legend for the color-coded status, and includes clear instructions for generating/scanning QR codes and managing attendance
- The GUI was made using Tkinter's main components

Inputs and Outputs:

1. Inputs:

- **Student Name**
- **QR Code Data**
- **System Time**
- **User Interaction with the Buttons (Generate/Scan QR)**
- **Admin Login (using a password)**
- **Admin's Modifications to Attendance Record/s**

2. Outputs:

- **Generated QR Code Image**
- **Attendance Record Display**
- **Photographic Evidence (Snapshots)**
- **Filtered Search Results**
- **Modified/Deleted Attendance Records**
- **Exported Attendance Files**
- **System Notifications (Pop-up messages for errors, warnings, and confirmations)**

Logic Plan (Pseudocode):

START PROGRAM

INITIALIZE system variables, data storage, and LOAD Admin Password from file

DISPLAY main window with:
- Title, instructions, and legend
- Student Buttons (Generate QR, Scan QR)
- Admin Buttons (Login, Change Password)
- HIDDEN Admin Controls (Update Record, Delete Record, Logout)

- Search bar and Attendance table

```
WHILE program is running:

IF user clicks "Scan QR" THEN
OPEN camera

... (Validation checks) ...

IF valid QR detected THEN
CAPTURE current frame (Snapshot)
SAVE snapshot to date-stamped folder
DETERMINE punctuality status based on 7:30 AM threshold
RECORD attendance
DISPLAY student in table with color coding
END IF

CLOSE camera
END IF


IF user clicks "Admin Login" THEN
PROMPT for password

IF password matches THEN
SET is_admin = TRUE
SHOW Update, Delete, and Logout buttons
ELSE
DISPLAY "Access Denied" error
END IF
END IF


IF user is Admin AND clicks "Update Record" THEN
OPEN update form with selected record's data
VALIDATE new inputs (Regex check for Name and Time)
SAVE changes to memory
REFRESH table display
END IF
```

```
IF user is Admin AND clicks "Delete Record" THEN
CONFIRM deletion with user

IF confirmed THEN
REMOVE student from records and memory list
REFRESH table
UPDATE scan count
END IF
END IF


IF user clicks "Change Password" THEN
VERIFY current password

IF valid THEN
SAVE new password to admin_password.txt
END IF
END IF


IF user clicks "Logout" THEN
SET is_admin = FALSE
HIDE Update, Delete, and Logout buttons
END IF


IF user attempts to close program THEN

... (Check for data) ...

EXPORT final records to CSV
EXPORT text report
CLOSE window

END IF

END WHILE

END PROGRAM
```

II. Methodology

Project Methodology

This section explains how our QR code-based attendance system was developed, focusing on its processes, technologies, and design choices. It covers our program's modular structure, the use of QR codes for generating and scanning attendance, and how the user interface interacts with the program logic. Key design decisions are also discussed to show how the system ensures efficiency, accuracy, and ease of use. Overall, this methodology provides a clear overview of how the system works and how its components achieve our project's goals.

A. *Technology Used*

Our program integrates several technologies to achieve its functionality, with QR code technology as the core component. The qrcode library is used to generate QR codes, while cv2 (OpenCV) allows for real-time scanning and decoding through the system's webcam. This combination allows for fast, contactless, and automated attendance recording.

QR code technology was chosen due to its efficiency, low cost, and ease of implementation. QR codes are faster and more flexible than traditional barcodes, which is why they're now used everywhere, from restaurants and packaging to business cards and marketing campaigns (*FreeQRToolsTeam, 2025*). Unlike most systems, such as biometric systems, QR codes do not require specialized hardware, making them more accessible for classroom use. Additionally, the use of tkinter for the graphical user interface provides a lightweight and built-in solution for application development in Python. Other supporting technologies include json for structured data handling and csv for report generation, which contribute to organized and portable data management

B. *Core Features*

Our program follows a modular programming approach, the process of subdividing a computer program into separate sub-programs, which is good for reusability and ease of maintenance (*GeeksforGeeks, 2025*). The generate_qr() function handles QR creation by validating user input using regular expressions, encoding it into JSON format, and converting it into a QR code image. QR codes are effective for storing structured data in a compact, machine-readable format, making them suitable for attendance systems.

Meanwhile, the scan_qr() function manages real-time QR detection using a webcam and OpenCV's QRCodeDetector. QRCodeDetector is reliable, as it's capable of both reading a QR code from an image and live camera feed (*note.nkmk.me, 2022*). Upon successful detection, the save_snapshot() function is immediately triggered to capture a real-time image of the student, providing a photographic record for verification. The system then verifies the QR code using a predefined key before passing the extracted name to register_student(). This function prevents duplicate entries, determines attendance status through get_time_status_and_time(), and records the data for display. This ensures accurate and automated tracking of attendance with minimal user intervention.

Our system also includes a secure Admin Panel governed by admin_login() and admin_logout(), which restricts advanced functions behind a password-protected interface. Upon a successful login, the Person-in-Authority (PIA) can utilize update_record() and delete_record() to manually modify the attendance list, ensuring the data remains accurate even if user errors occur. Furthermore, change_password() allows for the modification of security credentials, which in this case is the password for the login.

Supporting functions enhance usability and data management. The filter_table() function enables real-time searching by updating displayed records based on user input, while export_csv() and export_text_report() allow data to be saved in structured formats. The on_close() function ensures automatic saving before exit, preventing data loss, and simple_prompt() improves user interaction through validated input dialogs. Together, these components create an efficient and user-friendly attendance system.

C. *Backend-Frontend Communication*

When it comes to backend-frontend communication, our program follows a basic frontend-backend interaction model. The frontend consists of the graphical user interface built with tkinter, where users interact with buttons, input fields, and the attendance table, while the admin also interacts with the admin buttons after logging in.

The backend is composed of the program logic written in Python, which processes user inputs and performs operations such as QR generation, scanning validation, and data storage. Communication between the frontend and backend occurs through function calls triggered by user actions. For example, when the

user clicks the “Scan QR” button, the frontend triggers the scan_qr() function, which processes the input and updates the interface accordingly.

D. *Key Design Decisions*

During the early stages of our program, we relied heavily on a terminal-based program. However, we eventually needed to implement a GUI for our program at some point, which is why transitioning from a terminal-based to a GUI-based application was a very notable design decision. We needed to make a rough outline of our GUI based on our current program at that time, and that also meant having to make necessary revisions to the code to make it suitable for the GUI implementation, in which we eventually went with using Tkinter for our GUI.

After the outline of the GUI was finished, we started fully implementing the GUI. We had to ensure that the user interface was designed to be simple and intuitive, with clearly labeled buttons and a structured table layout. Color-coded attendance statuses were also implemented to allow users to quickly identify student conditions without reading detailed text.

These design choices were key to making the attendance system work effectively. We used Tkinter to build a desktop-based application, which makes it easy to deploy and means the program doesn't need an internet connection to run. Tkinter is a built-in graphical user interface (GUI) toolkit in Python that allows developers to create windows, buttons, and other interface elements for desktop applications (*Best DevOps, 2025*). Because it is already included with Python and is simple to use, it was the most practical way to build a lightweight and reliable program for both the students and the administrator.

References:

FreeQRTools Team. (2025, October 26). *What Is a QR Code and How Does It Work? (Complete Guide 2025)*. Free QR Tools. https://freeqrtools.com/what-is-a-qr-code-and-how-it-works/

GeeksforGeeks. (2025). *Modular Approach in Programming*. https://www.geeksforgeeks.org/software-engineering

note.nkmk.me. (2022). *Detect and read QR codes with OpenCV in Python*. https://note.nkmk.me/en/python-opencv-qrcode/

Best DevOps. (2025). *Tkinter: Comprehensive Guide to Python's Standard GUI Library.* *https://www.bestdevops.com/tkinter-comprehensive-guide-to-pythons-standard-gui-library/*